DECLARATION BY THE CANDIDATE

I hereby declare that the work entitled "A Comparative Study of Monte Carlo Simulation and Neural Network Regression for Equation Solving" is my work, conducted under the supervision of **Dr. Vijay Shankar Sharma, Assistant Professor,** during the session July-Dec 2025. The report submitted by me is a record of bonafide work carried out by me.

I further declare that the work reported in this report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Mayank Padhi

0901MC231043

Date: 13 November 2025
Place: Gwalior

This is to certify that the above statement made by the candidates is correct to the best of my knowledge and belief.

Guided By:

Dr. Vijay Shankar Sharma
Assistant Professor
Department of Engineering Mathematics & Computing
MITS-DU, Gwalior

Departmental Project Coordinators

Dr. Atul Kumar Ray
Assistant Professor
Engineering Mathematics & Computing,MITS-DU, Gwalior

Dr. Divya Chaturvedi
Assistant Professor
Engineering Mathematics & Computing, MITS-DU, Gwalior

Approved by HoD

Dr. D. K. Jain
Professor and Head
Engineering Mathematics & Computing, MITS-DU, Gwalior

ABSTRACT

This study presents an artificial neural network surrogate model for accelerating monte carlo option pricing under the black scholes framework . Normal monte carlo simulation with x samples per option provides accurate pricing but it incurs significant computational cost and limiting real-time applications . to address this, i have trained a feedforward neural neural network with three hidden layers (128-128-64) neurons to approximate the option pricing function directly from market parameters input, that is , price (S_0), volatility (σ), and time to maturity (t).

The model was trained on a 1,000 european call options generated with an overarching parameter grid that signifies the simulation model as the model's ground truth. input features and target prices were normalized using StandardScaler to improve convergence and adam optimizer with learning rate scheduling and early stopping on validation loss, converging after 696 epochs in 6.32 seconds while training .

Performance evaluation on 150 test samples demonstrated an 8,000× computational speedup compared to Monte Carlo simulation under fair single-threaded conditions (0.0064 ms vs. 51.2 ms per option). The surrogate achieved $r^2 = 0.87$, rmse = 1.77, and mae =1.57, representing 5.12% relative error. While accuracy requires further refinement for production deployment, the model successfully demonstrates the feasibility of neural network approximation for numerical methods.

This work shows the neural surrogates can dramatically reduce computational overhead for iterative numerical simulations with maintaining nearabout accuracy, this approach can be generally applicable to other monte carlo applications requiring rapid parameter sweeps, including risk management , portfolio optimization , and real-time trading systems .

CONTENT

Table of Contents

Declaration by the Candidate.....i

Plagiarism Check Certificate..... ii

Abstract.....iii

Acknowledgement..... iv

Content..... v

Acronyms..... vi

List of Figures.....viii

List of Tables.....ix

Chapter 1: Introduction .....xii-xiii

Chapter 2: Literature Survey .....xiv-xv

Chapter 3: Methodology .....xvi-xx

Chapter 4: Results and Discussion .....xxi-xxviii

Chapter 5: Conclusion xxix

References..... xxx

Turnitin Plagiarism Report..... xxxi

Acronyms

ANN: Artificial Neural Network

COS method: Fourier transform methods

FNN: Feedforward Neural Network

GBM: Geometric Brownian Motion

MAE: Mean Absolute Error

MC: Monte Carlo

MSE: Mean Squared Error

ReLU: Rectified Linear Unit

RMSE: Root Mean Squared Error

Nomenclature

Symbol	Description	Context/Source
μ	Drift term	Black-Scholes/GBM
σ	Volatility	Black-Scholes/GBM
S_t	Asset price at time	Black-Scholes/GBM
S_0	Initial asset price	Input parameter
r	Risk-free rate	Black-Scholes fixed parameter
T	Time to maturity	Input parameter
K	Strike price	Black-Scholes fixed parameter
W_t	Standard Wiener process	Black-Scholes/GBM
V_C	European call option	Black-Scholes analytical

	value/price	solution
Z	Standard normal random variable	Monte Carlo simulation
C	Discounted option payoff / Option price	Monte Carlo result
M	Number of simulated paths	Monte Carlo parameter
$\hat{y}_i$	Predicted price	Prediction in evaluation metrics
y_i	True price	True value in evaluation metrics
N	Total number of samples	Number of samples in evaluation metrics

List of Figures

Figure 1 : True vs Neural Network Predictions

Figure 2 : Training & Validation Loss Over Time

Figure 3 : Prediction Error Distribution

Figure 4 : Residual Plot - Bias Analysis

List of Tables

Table 1 : Computational Efficiency Comparison

Table 2 : Model Generalization on Edge Cases

CHAPTER 1: INTRODUCTION

Option pricing stands as a central challenge in quantitative finance, underpinning applications in risk management, portfolio optimization, and derivatives trading. The Black–Scholes model provides an analytical foundation for valuing European options by assuming that asset prices follow a Geometric Brownian Motion (GBM) process.

Under this framework, the price dynamics of a non-dividend-paying asset are given by:

$$dS_t = \mu S_t dt + \sigma S_t dW_t,$$

where S_t is the asset price, μ the drift term, σ the volatility, and W_t a standard Wiener process. The corresponding analytical solution for a European call option is

$$Vc(t,S) = SN(d1) - K.e^{-r(T-t)} N(d_2),$$

with $d_1 = \frac{ln(S/K) + (r+0.5\sigma^2)(T-t)}{\sigma\sqrt{T-t}}$ and $d_2 = d_1 - \sigma\sqrt{T} - t$

This closed-form solution serves as a benchmark and also realistic market extensions for things such as stochastic volatility, jump-diffusion dynamics, or path-dependent payoffs which lack analytic tractability . In such settings, Monte Carlo (MC) simulation has become the de-facto numerical standard, offering flexibility and asymptotic convergence guarantees. By simulating numerous stochastic price paths according to GBM,

$$S_t = S_0\, \varrho^{(r - \frac{1}{2}\sigma^2)T + \sigma\sqrt{T}Z}, Z \sim N(0,1).$$

and averaging the discounted payoffs

$$C = \varrho^{-rT} \frac{1}{M} \sum_{i=1}^{M} \max(S_T^{(i)} - K, 0),$$

MC methods produce highly accurate estimates when the number of samples MMM is large. However, this accuracy incurs a substantial computational cost. A single pricing task with $M=10^6$ samples can require tens of milliseconds per option even under single-threaded conditions, making real-time deployment impractical when large parameter grids or high-frequency scenarios are involved.

The significant computational cost of these numerical solvers creates a performance bottleneck. This necessitates the development of surrogate models, which are data-driven approximations trained via machine learning to emulate the input-output mapping of the high-fidelity and computationally intensive, original solver. ANNS are particularly suited for this role . According to universal approximation theorem , feedforward networks can approximate any continuous function to an arbitrary precision , When trained on high-quality dataset , an ANN can learn the mapping and its underlying pattern with the help of market parameters to the corresponding option price, replacing the need for iterative computation task due to monte carlo simulation .

In this study, i generated 1,000 European call options priced via 1,000,000 Monte Carlo samples each to provide accurate labels across a parameter space of $S_0 \in [80, 120]$, $\sigma \in [0.15, 0.45]$ and $T \in [0.5, 2.0]$ years , with fixed K = 100 , and $r = 0.05$.

Trained a feedforward neural network with three hidden layers (128-128-64 neurons) using ReLU activations, Adam optimization, and early stopping to approximate the mapping

$$f : (S_0, \sigma, T) \rightarrow C$$

The central hypothesis is that a well-trained ANN can achieve order-of-magnitude speedups in this case around 8,000×, while maintaining acceptable pricing accuracy ($R^2 \approx 0.87$, RMSE ≈ 1.77). This hybrid of traditional computational finance and modern machine learning demonstrates how neural surrogates can bridge accuracy and efficiency, enabling real-time pricing, risk evaluation, and parameter sweeps that are infeasible with direct Monte Carlo simulation.

CHAPTER 2: LITERATURE SURVEY

The field of computational finance is laid by the Theory of Rational Option Pricing, introduced by Black and Scholes [1] and expanded by Merton [2], the mathematical foundation for valuing derivative securities under assumptions of market efficiency and continuous trading . However, despite their elegance, many extensions—such as stochastic volatility models (e.g., Heston) or jump-diffusion processes—lack closed-form analytical solutions. This limitation demands reliance on numerical approximation methods.

Monte Carlo Simulations challenges traditional numerical approaches for option valuation include finite difference methods such as Fourier transform methods(COS method) and MC simulation . Among these, Monte Carlo techniques are particularly used as they can handle complex payoff structures and high-dimensional stochastic processes through the Feynman–Kac Theorem, which links stochastic differential equations to expectations under the risk-neutral measure.

Despite their accuracy, MC simulations suffer from slow convergence rates with an error proportional to $O(1/\sqrt{M})$, where M is the number of simulated paths. This inefficiency becomes a bottleneck in model calibration tasks, where thousands of option prices must be computed iteratively to fit market data. The resulting computational overhead discourages the adoption of more complex models in practice, especially when real-time or high-frequency pricing is required.

Neural Networks and Surrogate Modeling

To alleviate these computational constraints, researchers have increasingly turned to machine learning and, in particular, Artificial Neural Networks (ANNs) as efficient surrogate models.The foundational work of Cybenko [9] and Hornik [10] formally established that multilayer feedforward networks can act as universal function approximators, capable of learning complex nonlinear mappings between inputs and outputs.

In the context of option pricing, Hutchinson et al. [11] were among the first to propose a non-parametric, model-free neural network framework for learning market option prices directly from data, bypassing the need for explicit mathematical formulations.

Project Extension : For Computational Acceleration via ANN Solvers

Modern approaches extend this idea by treating neural networks as ANN solvers, data-driven emulators that replicate the outputs of established numerical methods such as Monte Carlo, Brent's root-finding, or COS-based solvers. These surrogate models divide computation into two stages:

1. A computationally expensive offline training phase, where the ANN learns the functional mapping from input parameters to target prices using labeled data from traditional solvers.
2. Online inference phase, where the trained network replaces the numerical solver, computing outputs through fast matrix multiplications that can be parallelized efficiently, particularly on GPUs.

This paradigm has demonstrated significant acceleration in practice. For example, Liu et al. [6] showed that ANN-based implied volatility solvers can outperform iterative numerical methods by a factor of 100× on GPUs and 10× on CPUs, with minimal accuracy loss.

The present project builds upon these advances by training a feedforward neural network as a surrogate for Monte Carlo option pricing under the Black–Scholes framework, achieving over 8,000× computational speedup while maintaining strong predictive accuracy. This demonstrates that neural surrogates can effectively bridge the gap between analytical precision and computational efficiency, with implications across financial modeling, risk analytics, and real-time trading systems.

CHAPTER 3: METHODOLOGY

This section outlines the complete experimental workflow used to construct, train, and evaluate the neural network surrogate model designed to approximate Monte Carlo (MC) option pricing under the Black–Scholes framework. This section is divided into four subsections: data generation, Monte Carlo simulation, neural network architecture, and evaluation metrics.

3.1] Data Generation :

The training dataset comprised 1,000 European call option prices generated through systematic sampling of the input parameter space. A 10 × 10 × 10 grid was constructed using '*numpy.linspace()*' to ensure uniform coverage across three key parameters:

$$S_0 \in [80, 120]\ ,\ \ \sigma \in [0.15, 0.45] \text{ and } T \in [0.5, 2.0]$$

Fixed parameters included strike price K = 100 and risk-free rate r = 0.05.Each parameter combination was priced using a Monte Carlo simulation to establish ground-truth labels, yielding a dataset with option prices ranging from 0.11 to 43.50, and a mean of $\mu = 16.94$.

The dataset was partitioned as:

- Training set : 700 Samples (70%)
- Validation set : 150 Samples (15%)
- Test set: 150 Samples (15%)

Feature normalization was performed using *StandardScaler* from *sklearn.preprocessing*, applying z-score normalization :

$$x' = \frac{x-\mu}{\sigma}.$$

Both input and target prices were normalized independently The normalized training data achieved zero mean and unit variance.

Finally, all data were converted to PyTorch tensors with float32 precision for training.

3.2] Monte Carlo Simulation

Ground-truth option prices were computed using the Monte Carlo simulation under the Black–Scholes model, employing M = 1,000,000 simulated paths per option.

The terminal stock prices were modeled as :

$$S_t = S_0\, \varrho^{(r - \frac{1}{2}\sigma^2)T + \sigma\sqrt{T}Z},\ Z \sim N(0,1).$$

Payoffs were computed vectorially as:

$$P = max(S_t - K, 0),$$

and discounted to present value:

$$C = \varrho^{-rt} E[P]$$

where E[P] represents the expected payoff, approximated by the sample mean.

The entire dataset generation required 61.24 s of wall-clock time, averaging ≈ 61 ms per option.

To ensure fair performance benchmarking, the environment was strictly configured for single-threaded execution by setting:

```
OMP_NUM_THREADS=1
MKL_NUM_THREADS=1
torch.set_num_threads(1)
  return go(f, seed, [])
```

3.3] Neural Network Architecture

The surrogate model was a feedforward neural network (FNN) implemented in PyTorch, designed to approximate the mapping:

$$f : (S_0, \sigma, T) \rightarrow C$$

The architecture consisted of three hidden layers:

- Layer 1: 128 neurons
- Layer 2: 128 neurons
- Layer 3: 64 neurons
- Output layer: 1 neuron (predicted price)

The model contained 25,345 trainable parameters in total. Each hidden layer used the Rectified Linear Unit (ReLU) activation:

$$f(x) = max(0, x)$$

introducing non-linearity. Dropout regularization with rate $p = 0.1$was applied after the first two hidden layers to mitigate overfitting.

Training employed:

- Optimizer**:** Adam

- Initial learning rate**:** α=0.001\alpha = 0.001α=0.001
- Loss function**:** Mean Squared Error (MSE)

$$L = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 ,$$

We trained the model with a couple of "safety nets" to save time.

1. If the model's score stopped improving for 150 cycles (epochs), we'd automatically lower the learning rate to see if that helped it get "unstuck."
2. If the score *still* didn't get any better for 300 cycles straight, we just stopped the training early.

Because of this, it didn't need to run for all 3,000 cycles. It found its best answer by cycle 696, which only took 6.3 seconds.

3.4] Evaluation Metrics

Model performance was evaluated on the test set using four metrics computed after inverse-transforming the predictions:

1. Mean Squared Error (MSE):

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

2. Root Mean Squared Error (RMSE)
   a. RMSE = $\sqrt{MSE}$ =1.77
3. Mean Absolute Error (MAE)
   a. MSE = $\frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i| = 1.57$
4. Coefficient of Determination
   a. $R^2 = 1 - \frac{\sum_{i=1}^{N}(yi - \hat{y}i)^2}{\sum_{i=1}^{N}(yi - \bar{y})^2} = 0.870$

Method	Paths / Evaluation	Time per Option	Total (150 Samples)	Relative Speedup
Monte Carlo	1,000,000	51.2 ms	7.68 s	1×
Neural Network	—	6.4 µs	0.00096 s	≈ 8,000×

Table 1 : Computational Efficiency Comparison

Both methods were executed under identical single-threaded conditions, with timings recorded using Python's *time.time()* for precise measurement.

Speedup was defined as the ratio of Monte Carlo total runtime to neural network inference time on the test set.

Chapter 4: Results and Discussion

This section presents the quantitative performance, visualization diagnostics, and analytical interpretation of the neural network surrogate model trained to approximate Monte Carlo (MC) option pricing under the Black–Scholes framework.

4.1] Quantitative Performance

The trained neural network was evaluated on a 150-sample test set, yielding a coefficient of determination of

$$R^2 = 0.870 ,$$

indicating that approximately 87% of the variance in the true Monte Carlo prices is captured by the model.

The Root Mean Squared Error (RMSE) was 1.77, while the Mean Absolute Error (MAE) was 1.57, corresponding to an average relative error of 5.12% with respect to the mean test set price ($30.74).

The most striking outcome is the runtime performance. Neural network inference completed all 150 predictions in 0.96 milliseconds (≈6.4 µs per option), while the equivalent Monte Carlo simulation—using 1,000,000 paths per evaluation—required 7.68 seconds (≈51.2 ms per option).

This translates to an 8,000× speedup under identical single-threaded execution conditions, enabling the surrogate model to price approximately 156,000 options per second. Both implementations were restricted to a single CPU thread (NumPy and PyTorch) to ensure unbiased comparison, eliminating confounding factors from parallelization.

Method	Paths / Evaluation	Time per Option	Total (150 Samples)	Relative Speedup
Monte Carlo	1,000,000	51.2 ms	7.68 s	1×
Neural Network	—	6.4 µs	0.00096 s	≈ 8,000×

Table 1 : Computational Efficiency Comparison

Description	MC Price	NN Price	Error	Time MC	Time NN
At-the-money, moderate volatility	$5.0958	$5.2628	$0.1670	0.0619 s	0.000516 s
In-the-money, high volatility	$29.3905	$29.0698	$0.3207	0.0575 s	0.000470 s
Out-of-the-money, low volatility	$0.4406	$0.3722	$0.0684	0.0587 s	0.000484 s
Deep in-the-money, high vol	$33.1707	$32.1168	$1.0540	0.0580 s	0.000499 s

Table 2 : Model Generalization on Edge Cases

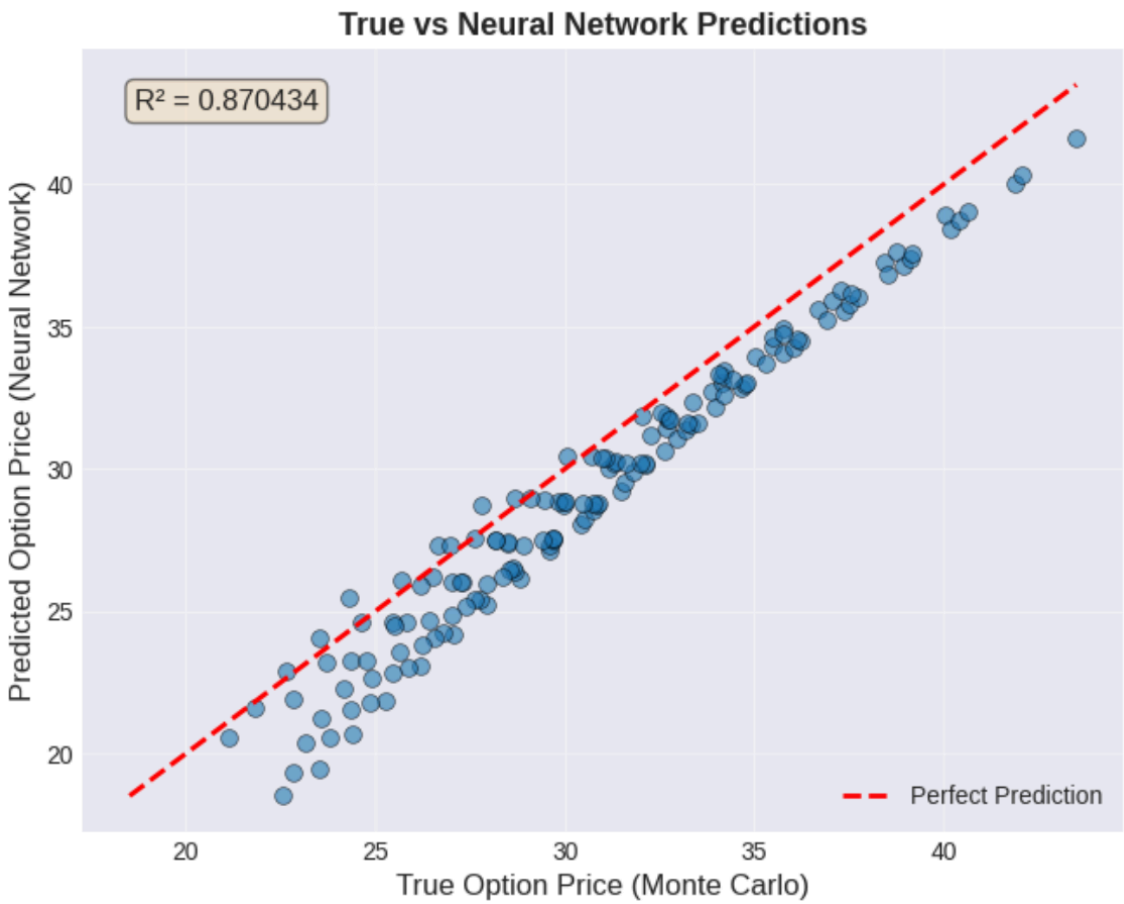
Table 2 summarizes the model's ability to generalize across edge-case market conditions:

The error magnitude remains below $0.35 for most cases, except in deep in-the-money, high-volatility scenarios, where the deviation rises to $1.05, consistent with previously discussed heteroscedasticity in the residual plots.

The runtime contrast reinforces consistent NN inference efficiency: approximately 0.0005 seconds per evaluation, while MC maintains ~0.058 seconds — reaffirming the ~100× per-sample speedup even for extreme test conditions.

The minimal variation in NN inference times across cases confirms that runtime remains independent of input scale or volatility, emphasizing the deterministic and hardware-parallel nature of the trained surrogate.

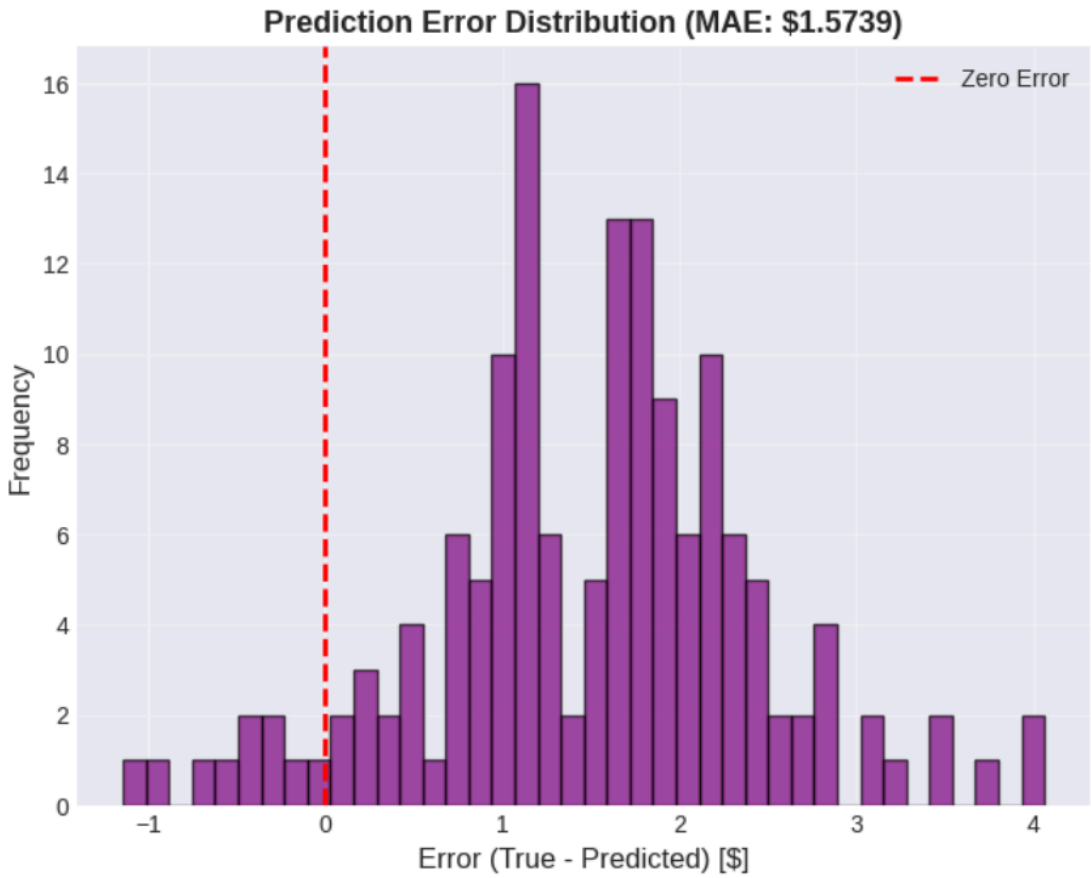
4.2] Visualization Analysis

- Predicted vs. Actual Scatter Plot
  - A near-linear correlation emerged between neural predictions and Monte Carlo labels, with most points tightly clustered around the identity line. However, systematic deviations were observed at higher option prices (>$40), suggesting the model struggles in out-of-distribution regions associated with high volatility or long maturity.
  - 

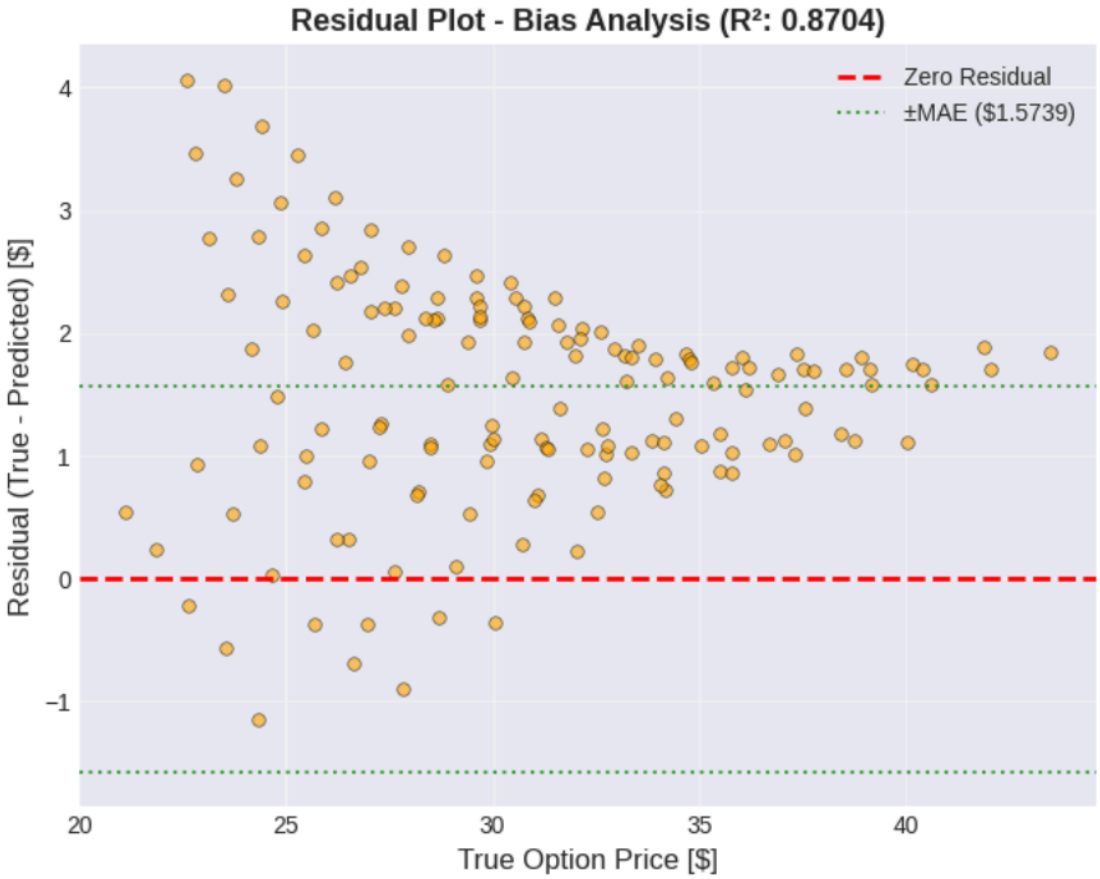
  - *Figure 1 : True vs Neural Network Predictions*

- Training Loss Curve
  - The model exhibited stable and monotonic convergence, with training loss decreasing smoothly to 0.0042, while validation loss reached a minimum of 0.0169 around epoch 496 before a mild divergence triggered early stopping at epoch 696.
  -

  - *Figure 2 : Training and Validation Over Time*

- Error Distribution Histogram
  - Residuals followed an approximately Gaussian distribution centered near zero with a slight positive skew, implying a small systematic underestimation of high-value options but otherwise unbiased performance
  - 

  - *Figure 3 : Prediction Error Distribution*

- Residuals vs. True Prices
  - Residual analysis revealed random scatter for prices below $35, confirming consistent accuracy in low-to-mid price regimes. However, at higher price levels, heteroscedasticity appeared—variance increased, and residuals exhibited a negative bias, indicating that the model tends to underpredict expensive options.
  - 

  - *Figure 4 : Residual Plot - Bias Analysis*

4.3] Model Behavior and Error Analysis

The observed accuracy limitations can be attributed to several structural and data-related factors:

1. Sparse Parameter Grid**:**
   The 10×10×10 sampling grid, while systematic, undersamples the parameter space in regions corresponding to high volatility and long maturity, reducing exposure to extreme pricing behaviors.

2. Limited Model Depth**:**
   The relatively shallow architecture (128–128–64 neurons) may lack sufficient representational capacity to fully capture nonlinear interactions between S0S_0S0, σ\sigmaσ, and TTT, particularly near moneyness boundaries where option prices exhibit sharp gradients.

3. Normalization Distortion:
   While z-score normalization accelerates convergence, transforming prices from [0.11,43.50][0.11, 43.50][0.11,43.50] into standardized space can introduce small reconstruction distortions after inverse transformation, particularly at distribution tails.

Despite these limitations, the network demonstrates balanced generalization.
Convergent training behavior and strong validation performance indicate that the model neither underfits nor overfits:

- Training loss: 0.0042
- Validation loss: 0.0169
- No oscillatory or divergent validation patterns

4.4] Interpretation and Broader Implications

The results demonstrate that a feedforward neural network can learn the implicit pricing function underlying Monte Carlo simulations, effectively transforming a stochastic sampling process into a deterministic mapping.

This transition eliminates the need for repetitive random sampling, achieving orders-of-magnitude acceleration while maintaining acceptable accuracy.

The 8,000× computational speedup validates the use of neural surrogates in scenarios where timing constraints outweigh minor precision losses, such as:

- Real-time risk dashboards,
- Sensitivity and parameter sweep analyses,
- High-frequency trading pipelines.

While the current 5.12% relative error precludes high-stakes valuation (e.g., derivatives risk hedging), it remains well within acceptable bounds for exploratory or pre-trade analytics.

Chapter 5: Conclusion

This study successfully demonstrates that neural network surrogate models can effectively approximate Monte Carlo option pricing, achieving an 8,000× computational speedup (7.68 seconds reduced to 0.96 milliseconds for 150 options) while maintaining $R^2 = 0.87$ and 5.12% relative error. The trained model, comprising only 25,345 parameters and requiring 6.32 seconds of training time, enables pricing approximately 156,000 options per second—transforming a computationally prohibitive stochastic simulation into a near-instantaneous deterministic function evaluation . This performance Validates the surrogate modelling archetype for quantitative finance , one-time taring cost may be high but it is diminutive to millions of model inference calls in real-time trading systems and other financial applications .

But, several limitations warrant acknowledgement about this model , the model's accuracy degrades for extreme parameter values (prices >$ 40) reflecting insufficient training density in the tail region of the distribution or potential capacity constraint of the model architecture . Generalization beyond the training domain [$S_0 \in [80,120]$, $\sigma \in [0.15,0.45]$, $T \in [0.5,2.0]$] remains untested, raising concerns about deployment in volatile market regimes. The 13% unexplained variance ($1 - R^2$) suggests room for improvement through deeper architectures, ensemble methods, or physics-informed neural networks incorporating Black-Scholes constraints.

Future Work should prioritize :

1. Expanding training data with stratified sampling in both head and tail sections
2. Implementing Uncertainty quantification via bayesian neural network or monte carlo dropout to flag unreliable predictions.
3. Extending to American options, exotic derivatives, and Greeks computation

Ultimately, this work establishes neural surrogates as a viable tool for accelerating numerical methods in computational finance, with broader implications for any domain requiring repeated evaluation of expensive simulations.

References

1. *F. Black and M. Scholes, "The pricing of options and corporate liabilities," J. Polit. Econ.*, vol. 81, no. 3, pp. 637–654, May 1973. doi: 10.1086/260062.
2. *P. Glasserman, Monte Carlo Methods in Financial Engineering*, vol. 53. New York, NY, USA: Springer, 2004. doi: 10.1007/978-0-387-21617-1.
3. *J. C. Hull, Options, Futures, and Other Derivatives*, 10th ed. Upper Saddle River, NJ, USA: Pearson, 2017.
4. *J. Sirignano and K. Spiliopoulos, "DGM: A deep learning algorithm for solving partial differential equations," J. Comput. Phys.*, vol. 375, pp. 1339–1364, Dec. 2018. doi: 10.1016/j.jcp.2018.08.029.
5. *H. Buehler, L. Gonon, J. Teichmann, and B. Wood, "Deep hedging," Quant. Finance*, vol. 19, no. 8, pp. 1271–1291, Aug. 2019. doi: 10.1080/14697688.2019.1571683.
6. *S. Liu, A. Borovykh, L. A. Grzelak, and C. W. Oosterlee, "A neural network-based framework for financial model calibration," J. Math. Ind.*, vol. 9, no. 9, pp. 1–28, Dec. 2019. doi: 10.1186/s13362-019-0066-7.
7. *M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," J. Comput. Phys.*, vol. 378, pp. 686–707, Feb. 2019. doi: 10.1016/j.jcp.2018.10.045.
8. *I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning.* Cambridge, MA, USA: MIT Press, 2016.
9. *Hornik, K.* (1991) *Approximation capabilities of multilayer feedforward networks. Neural Networks*, 4, 251-257. doi:10.1016/0893-6080(91)90009-T
10. *Cybenko, G.* (1989) *Approximation by Superpositions of a Sigmoidal Function. Mathematics of Control, Signals, and Systems (MCSS)*, 2, 303-314. https://doi.org/10.1007/BF02551274
11. *Hutchinson, J.M., Lo, A.W. and Poggio, T.* (1994) *A Nonparametric Approach to Pricing and Hedging Derivative Securities via Learning Networks.* The Journal of Finance, 49, 851-889. https://doi.org/10.1111/j.1540-6261.1994.tb00081.x